DataStore

El formulario ya guarda el nombre, el correo y la casilla en `RegistroViewModel`. Esa ficha sigue ahí si rotas el teléfono, porque el `ViewModel` sigue vivo. Esta guía agrega dos cosas:

- Un estado que vive en la pantalla: un contador de intentos, primero con `remember` y después con `rememberSaveable`, y un texto calculado con `derivedStateOf`.
- Una copia del nombre y del correo en el teléfono, con DataStore. Esa copia sigue ahí aunque se cierre la app.

Rotar el teléfono es pasarlo de vertical a horizontal, o al revés. Android cierra la pantalla y la vuelve a crear. Sirve para ver qué dato se pierde y cuál se queda.

La ruta `register_page` y el `NavHost` no se tocan. Los colores, el movimiento y el loader van en la guía de animaciones.

Dónde vive cada dato

Dato	Dónde se guarda	Qué lo borra
Texto de los campos	`RegistroViewModel`	Cerrar la app
Contador con `remember`	La pantalla, mientras está dibujada	Rotar el teléfono o salir de la pantalla
Contador con `rememberSaveable`	La pantalla, y además el sistema lo anota	Cerrar la app
“Listo para enviar”	No se guarda: se calcula	Cambia solo cuando cambian los campos
Nombre y correo en DataStore	Un archivo del teléfono	Desinstalar la app, o guardar otros datos encima

1. Dependencias

En `app/build.gradle.kts`, dentro de `dependencies`, agrega esto y sincroniza Gradle:

```
implementation("androidx.datastore:datastore-preferences:1.1.7")
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.11.0")
```

- `datastore-preferences` guarda pares clave-valor, como el nombre y el correo. Es el reemplazo de `SharedPreferences`: la escritura es asíncrona y no se hace en el hilo que dibuja la pantalla.
- `lifecycle-viewmodel-ktx` trae `viewModelScope`. Ese alcance lanza las lecturas y escrituras del DataStore, y las cancela si el `ViewModel` se destruye.

2. Un contador con remember

En `RegisterScreen`, junto a las otras variables de arriba de la `Column`, agrega el contador:

```
var intentos by remember { mutableStateOf(0) }
```

Imports:

```
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
```

`remember` guarda un valor mientras esta función siga en pantalla. `mutableStateOf(0)` crea un estado observable que parte en 0. `by` deja escribir `intentos` e `intentos++` en vez de `intentos.value`.

Cada vez que `intentos` cambia, Compose vuelve a ejecutar `RegisterScreen` y dibuja el número nuevo. El estado es el dato. La función composable es la descripción de cómo se ve ese dato.

Arriba del botón Registrarse, muestra el número:

```
Text("Intentos: $intentos")
```

Y en el `onClick` de ese botón, antes del `if`, suma uno:

```
intentos++
```

El botón queda así:

```
Button(
    onClick = {
        intentos++
        if (viewModel.validar()) {
            scope.launch {
                snackbarHostState.showSnackbar("Formulario enviado correctamente")
            }
        } else {
            scope.launch {
                snackbarHostState.showSnackbar("Errores detectados. Revisa los campos.")
            }
        }
    }
) {
    Text("Registrarse")
}
```

Abre registro, pulsa el botón varias veces y rota el teléfono. El número vuelve a 0. `remember` vive en la composición, y al rotar esa composición se crea de nuevo.

3. El mismo contador con rememberSaveable

Cambia la línea del contador por esta:

```
var intentos by rememberSaveable { mutableStateOf(0) }
```

Import:

```
import androidx.compose.runtime.saveable.rememberSaveable
```

`rememberSaveable` hace lo mismo que `remember` y, además, le entrega el valor al sistema para que lo restaure después de rotar. Un `Int` se puede guardar así, sin escribir un conversor.

Pulsa el botón y rota el teléfono: el número se queda. Cierra la app desde la lista de recientes y vuelve a abrirla: el contador parte en 0. Sobrevive a la rotación. Se pierde al cerrar la app.

4. Un valor derivado con derivedStateOf

El texto “Listo para enviar” sale de tres datos que ya existen: el nombre, el correo y la casilla. No hace falta guardarlo en la ficha.

Junto al contador:

```
val puedeEnviar by remember {
    derivedStateOf {
        estado.nombre.isNotBlank() &&
            estado.correo.contains("@") &&
            estado.aceptaTerminos
    }
}
```

Import:

```
import androidx.compose.runtime.derivedStateOf
```

`derivedStateOf` calcula un valor nuevo a partir de otros estados. El cálculo corre cuando cambia el nombre, el correo o la casilla. Si la pantalla se redibuja por el contador, y esos tres siguen igual, el resultado se reutiliza.

`puedeEnviar` es un `Boolean`. Es `true` solo cuando hay nombre, el correo trae `@` y la casilla está marcada. No reemplaza a `validar()`: `validar()` arma los mensajes de error al pulsar el botón. Esto solo sirve para mostrar un aviso mientras la persona escribe.

Debajo del contador:

```
Text(if (puedeEnviar) "Listo para enviar" else "Faltan datos")
```

Escribe el nombre, un correo con `@` y marca la casilla. El texto pasa a “Listo para enviar” sin pulsar el botón. Borra el `@` y vuelve a “Faltan datos”.

5. Guardar el nombre y el correo

DataStore escribe en un archivo del teléfono. La app puede cerrarse y, al abrir el registro otra vez, los campos vuelven a llenarse.

La ficha del formulario gana dos datos para la interfaz: si se está guardando y si ya se guardó. La guía de animaciones los usa para mover el botón y mostrar el loader.

5.1 La ficha

En `RegistroUiState`, agrega estos dos campos al final:

```
data class RegistroUiState(
    val nombre: String = "",
    val correo: String = "",
    val aceptaTerminos: Boolean = false,
    val errores: RegistroErrores = RegistroErrores(),
    val guardando: Boolean = false,
    val guardado: Boolean = false
)
```

`guardando` es `true` mientras el archivo se está escribiendo. `guardado` es `true` cuando esa escritura terminó. Los dos parten en `false`.

5.2 El archivo de preferencias

Crea el paquete `cl.duoc.myapplication.data` y el archivo `RegistroPreferences.kt`:

```kotlin
package cl.duoc.myapplication.data

import android.content.Context
import androidx.datastore.core.DataStore
import androidx.datastore.preferences.core.Preferences
import androidx.datastore.preferences.core.edit
import androidx.datastore.preferences.core.stringPreferencesKey
import androidx.datastore.preferences.preferencesDataStore
import kotlinx.coroutines.flow.first

private val Context.registroDataStore: DataStore<Preferences> by preferencesDataStore(
    name = "registro"
)

data class DatosGuardados(
    val nombre: String,
    val correo: String
)

class RegistroPreferences(private val context: Context) {
    private val claveNombre = stringPreferencesKey("nombre")
    private val claveCorreo = stringPreferencesKey("correo")

    suspend fun leer(): DatosGuardados {
        val prefs = context.registroDataStore.data.first()
        return DatosGuardados(
            nombre = prefs[claveNombre] ?: "",
            correo = prefs[claveCorreo] ?: ""
        )
    }

    suspend fun guardar(nombre: String, correo: String) {
        context.registroDataStore.edit { prefs ->
            prefs[claveNombre] = nombre
            prefs[claveCorreo] = correo
        }
    }
}
```

`preferencesDataStore(name = "registro")` crea el archivo `registro`. La línea va afuera de la clase, pegada al `Context`, y el archivo solo se crea una vez.

`stringPreferencesKey` es el nombre de cada valor dentro de ese archivo. `"nombre"` y `"correo"` son las claves.

`data` es un `Flow`: una secuencia que entrega el contenido del archivo. `first()` espera la primera entrega y con eso alcanza para leer una vez al abrir la pantalla. `?: ""` cubre la primera vez, cuando la clave todavía no existe.

`edit` escribe. Es una función suspendida: hay que llamarla desde una corrutina. DataStore aplica el cambio en segundo plano y deja el archivo consistente.

`DatosGuardados` junta las dos lecturas para que el `ViewModel` reciba un solo objeto.

5.3 El ViewModel lee al abrirse y escribe al guardar

`RegistroViewModel` pasa a ser un `AndroidViewModel`. Así recibe la `Application` y, con ella, el `Context` que pide DataStore. `viewModel()` en la pantalla sigue igual: la actividad ya sabe crear

un `AndroidViewModel`.

Reemplaza `viewmodels/RegistroViewModel.kt` por esto:

```kotlin
package cl.duoc.myapplication.viewmodels

import android.app.Application
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.viewModelScope
import cl.duoc.myapplication.data.RegistroPreferences
import cl.duoc.myapplication.model.RegistroErrores
import cl.duoc.myapplication.model.RegistroUiState
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

class RegistroViewModel(application: Application) : AndroidViewModel(application) {
    private val preferences = RegistroPreferences(application)
    private val _estado = MutableStateFlow(RegistroUiState())
    val estado: StateFlow<RegistroUiState> = _estado.asStateFlow()

    init {
        viewModelScope.launch {
            val guardado = preferences.leer()
            _estado.update {
                it.copy(nombre = guardado.nombre, correo = guardado.correo)
            }
        }
    }

    fun onNombreChange(valor: String) {
        _estado.update {
            it.copy(
                nombre = valor,
                guardado = false,
                errores = it.errores.copy(nombre = null)
            )
        }
    }

    fun onCorreoChange(valor: String) {
        _estado.update {
            it.copy(
                correo = valor,
                guardado = false,
                errores = it.errores.copy(correo = null)
            )
        }
    }

    fun onAceptarTerminosChange(valor: Boolean) {
        _estado.update {
            it.copy(
                aceptaTerminos = valor,
                guardado = false,
                errores = it.errores.copy(aceptaTerminos = null)
            )
        }
    }

    fun validar(): Boolean {
        val actual = _estado.value
        val errores = RegistroErrores(
            nombre = if (actual.nombre.isBlank()) "El nombre es obligatorio" else null,
```

```
            correo = when {
                actual.correo.isBlank() -> "El correo es obligatorio"
                !actual.correo.contains("@") -> "El correo debe incluir @"
                else -> null
            },
            aceptaTerminos = if (!actual.aceptaTerminos) "Debes aceptar los términos" else
        null
        )
        _estado.update { it.copy(errores = errores) }
        return errores.nombre == null && errores.correo == null && errores.aceptaTerminos ==
        null
    }

    fun guardar() {
        val actual = _estado.value
        viewModelScope.launch {
            _estado.update { it.copy(guardando = true) }
            delay(800)
            preferences.guardar(actual.nombre, actual.correo)
            _estado.update { it.copy(guardando = false, guardado = true) }
        }
    }
}
```

`init` corre al crear el `ViewModel`. `leer()` trae lo último que quedó en el archivo y `copy` lo pone en la ficha. La pantalla observa esa ficha, así que los campos se llenan solos.

`viewModelScope` es el lugar desde donde el `ViewModel` lanza corrutinas. Si la pantalla se destruye de verdad, ese alcance se cancela.

`guardar` marca `guardando`, espera 800 milisegundos y después escribe. La espera deja el texto “Guardando…” el tiempo suficiente para leerlo. La escritura en DataStore, sola, termina antes. El nombre y el correo que se escriben son los que había al pulsar el botón, guardados en `actual`.

Al terminar, `guardando` vuelve a `false` y `guardado` queda en `true`. Si la persona cambia un campo, ese `guardado` vuelve a `false` para que el aviso de éxito se retire.

`validar()` sigue diciendo si los tres campos están bien.

5.4 El botón llama a guardar

En el `onClick` del botón Registrarse, el camino correcto deja de mostrar el snackbar de éxito y llama a `guardar()`:

```
Button(
    onClick = {
        intentos++
        if (viewModel.validar()) {
            viewModel.guardar()
        } else {
            scope.launch {
                snackbarHostState.showSnackbar("Errores detectados. Revisa los campos.")
            }
        }
    },
    enabled = !estado.guardando
) {
    Text(if (estado.guardando) "Guardando..." else "Registrarse")
}
```

`enabled = !estado.guardando` deja el botón quieto mientras la escritura no termina. Así no se lanzan dos guardados a la vez.

Debajo del botón, el aviso de que el archivo ya se escribió:

```
if (estado.guardado) {
    Text("Datos guardados en el teléfono")
}
```

Ese `if` muestra y esconde el texto de una vez. La guía de animaciones lo cambia por una aparición.

Llena el formulario, pulsa Registrarse y espera a ver “Datos guardados en el teléfono”. Cierra la app, ábrela y entra a registro: el nombre y el correo siguen en los campos. La casilla no se guarda. En esta guía DataStore solo persiste los dos textos.

Cómo queda RegisterScreen

Este es el archivo al terminar esta guía, sin las animaciones. Va en dos partes.

Parte 1, hasta el logo:

```
package cl.duoc.myapplication.ui

import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Checkbox
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.derivedStateOf
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import cl.duoc.myapplication.R
import cl.duoc.myapplication.navigation.Screen
import cl.duoc.myapplication.viewmodels.MainViewModel
import cl.duoc.myapplication.viewmodels.RegistroViewModel
import kotlinx.coroutines.launch

@Composable
fun RegisterScreen(mainViewModel: MainViewModel) {
    val viewModel: RegistroViewModel = viewModel()
    val estado by viewModel.estado.collectAsState()
    val snackbarHostState = remember { SnackbarHostState() }
    val scope = rememberCoroutineScope()
    var intentos by rememberSaveable { mutableStateOf(0) }
    val puedeEnviar by remember {
        derivedStateOf {
            estado.nombre.isNotBlank() &&
                estado.correo.contains("@") &&
                estado.aceptaTerminos
        }
    }

    Box {
        Image(
            painterResource(R.drawable.images),
            contentDescription = "",
            modifier = Modifier.fillMaxSize(),
            contentScale = ContentScale.Crop
        )

        Column(
```

```
            modifier = Modifier
                .fillMaxSize()
                .padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Spacer(modifier = Modifier.weight(1F))

            Image(
                painter = painterResource(R.drawable.logo),
                contentDescription = "Logo App",
                modifier = Modifier.padding(40.dp)
            )
```

Parte 2, el resto de la columna:

```
        OutlinedTextField(
            value = estado.nombre,
            onValueChange = viewModel::onNombreChange,
            label = { Text("Nombre completo") },
            isError = estado.errores.nombre != null,
            supportingText = {
                estado.errores.nombre?.let { mensaje ->
                    Text(mensaje)
                }
            },
            modifier = Modifier.fillMaxWidth()
        )

        OutlinedTextField(
            value = estado.correo,
            onValueChange = viewModel::onCorreoChange,
            label = { Text("Correo electrónico") },
            isError = estado.errores.correo != null,
            supportingText = {
                estado.errores.correo?.let { mensaje ->
                    Text(mensaje)
                }
            },
            modifier = Modifier.fillMaxWidth()
        )

        Row(verticalAlignment = Alignment.CenterVertically) {
            Checkbox(
                checked = estado.aceptaTerminos,
                onCheckedChange = viewModel::onAceptarTerminosChange
            )
            Text("Acepto los términos")
        }

        estado.errores.aceptaTerminos?.let { mensaje ->
            Text(mensaje)
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 16.dp))

        Text("Vista previa en tiempo real")
        Text("Nombre: ${estado.nombre}")
        Text("Correo: ${estado.correo}")
        Text("Términos aceptados: ${if (estado.aceptaTerminos) "Sí" else "No"}")

        Text("Intentos: $intentos")
        Text(if (puedeEnviar) "Listo para enviar" else "Faltan datos")

        Button(
            onClick = {
                intentos++
                if (viewModel.validar()) {
                    viewModel.guardar()
                } else {
                    scope.launch {
                        snackbarHostState.showSnackbar("Errores detectados. Revisa los
    campos.")
                    }
                }
            },
            enabled = !estado.guardando
        ) {
            Text(if (estado.guardando) "Guardando..." else "Registrarse")
```

```
            }

            if (estado.guardado) {
                Text("Datos guardados en el teléfono")
            }

            Spacer(modifier = Modifier.weight(1F))

            Button(
                onClick = { mainViewModel.navigateTo(Screen.Home) }
            ) {
                Text("Volver")
            }

            SnackbarHost(hostState = snackbarHostState)
        }
    }
}
```

Qué probar

1. Entra a registro y pulsa **Registrarse** sin llenar nada. El contador sube y los campos muestran su error.
2. Rota el teléfono, de vertical a horizontal. El contador se mantiene. El nombre que hayas escrito también, porque vive en el `ViewModel`.
3. Escribe nombre, un correo con `@` y marca la casilla. El texto pasa a “Listo para enviar” sin pulsar el botón.
4. Pulsa **Registrarse**. El botón dice “Guardando…” y no se puede pulsar. Después aparece “Datos guardados en el teléfono”.
5. Cierra la app desde la lista de recientes y ábrela de nuevo. En registro, el nombre y el correo siguen escritos. El contador vuelve a 0.